End-to-End Research Assistant with Multi-Source Synthesis and Alignment Analysis

AIMLCZG521 Conversational AI — Assignment 2, Problem Statement 1 · Final Report
Team: Prajwal Shetty K P (2025AE05434) · Poornima Prakash Patil (2025AE05411) · Prasanth C (2025AE05388) · Jay Kumar Patel (2025AE05473) — *contribution areas as listed in the notebook*

Summary. We built a research assistant that decomposes questions, calls academic, encyclopedia and news sources in parallel through a ReAct agent with six safety mechanisms, fuses evidence with credibility-weighted RRF and cross-encoder re-ranking, resolves conflicting claims by source credibility, writes cited syntheses, verifies and self-corrects its claims, attaches calibrated confidence, and records a full audit trail. It was evaluated on 10 complex held-out questions and 50 held-out Natural Questions with an LLM judge and a 1–5 rubric. The generator is Qwen2.5-14B-Instruct (Q4_K_M) running locally in LM Studio on an Intel Arc A770 GPU; every judge call uses Google gemini-3.5-flash-lite, a different model family. The lost-in-the-middle test uses gemini-3.1-flash-lite and the dense routing tier gpt-oss-120b on Groq. Costs are reported at GPT-4o prices, as the brief requires.

1. Headline results

Metric	Value
Synthesis quality (1–5)	4.62
Conflict resolution rate	100% (gold agreement 67%)
Citation accuracy	74%
Hallucination rate (before → after)	10.7% → 3.2%
Calibration error (ECE)	0.190
Task completion	100% complex · 98% NQ-50
Accuracy, 50 held-out NQ	86%
Cost per research query (GPT-4o pricing)	$0.0599 complex · $0.0380 simple
Latency P50 / P95 (complex)	65.7s / 88.4s
RAG Triad (CR / G / AR)	4.40 / 4.50 / 5.00
Judge vs human (Cohen's κ)	0.03 unweighted · 0.10 quadratic (30 human-labelled items)

2. Architecture & workflow

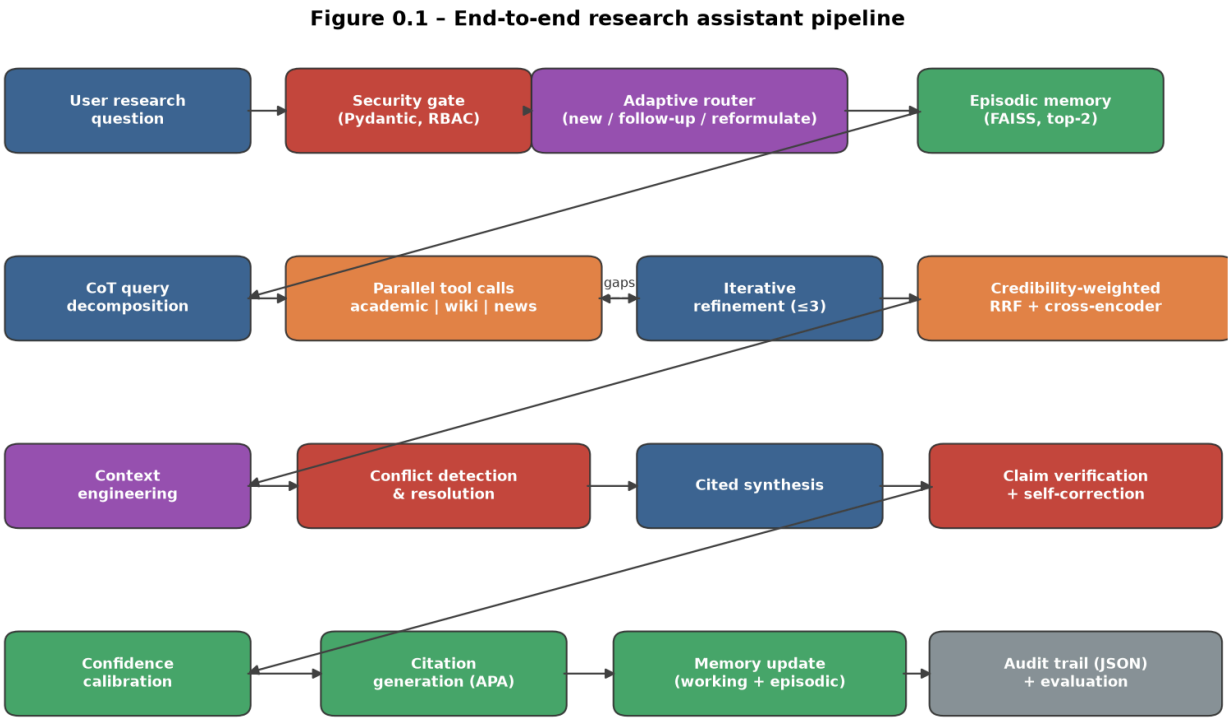


Figure 1 – Pipeline: Research Question → Decomposition → Multi-Source Function Calling → Evidence → Conflict Resolution → Synthesis → Citation → Memory Update

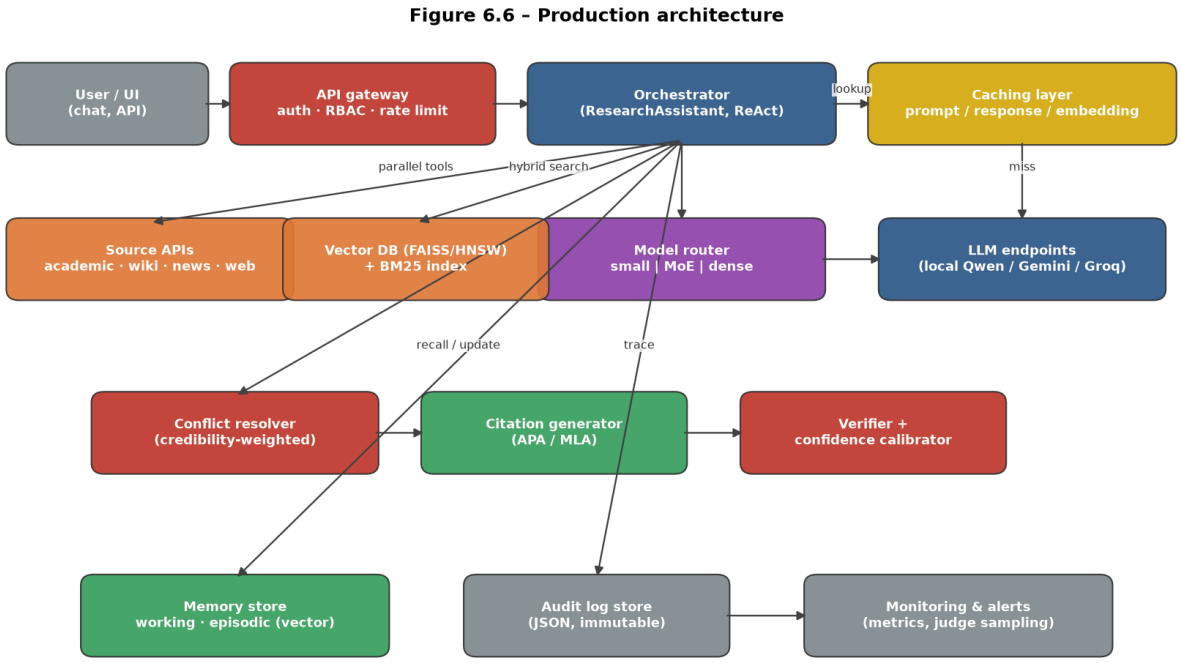


Figure 2 – Production architecture

3. Data

Natural Questions (Kwiatkowski et al., 2019; CC-BY-SA-3.0). 1,000 domain-filtered examples (seed 42), split into Task-1 simple (5), episodic seeds (30), retrieval-dev (100), held-out (50) and distractors (815). The questions

and gold answers come from nq_open and the gold passages from `sentence-transformers/natural-questions`. **Wikipedia**: 28 real articles. **Synthetic curated corpus**: 75 labelled documents (academic / news / forum) across 18 topics (8 dev, 10 held-out), with gold key facts and adjudicated conflicts. **50 synthetic episodic sessions** are built from dev data only, and held-out data never enters memory.

4. Task 1 — Prompting strategies

Technique	Correct %	Depth	Sources ref.	In tok	Out tok	Latency s	$ / 1k queries
Zero-Shot	75.00	0.90	2.40	729.80	90.35	6.67	5.004
Few-Shot	100.00	3.00	3.20	1402.80	218.45	11.24	10.291
CoT	90.00	5.25	3.60	808.80	371.20	15.73	9.612
ToT	90.00	3.20	2.75	3709.10	841.25	38.13	31.164
APE	100.00	3.10	3.40	769.80	248.80	11.57	7.581

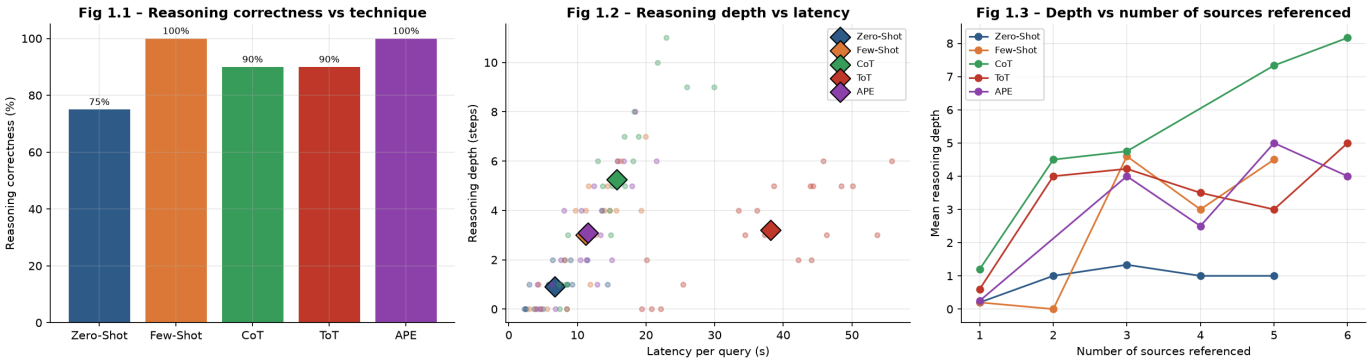


Figure 3 – Correctness, depth vs latency, depth vs sources

Which technique produces the most reliable multi-step reasoning? APE and Few-Shot were the most reliable: both were correct on **100 %** of the 20 questions, against CoT 90 %, ToT 90 % and Zero-Shot 75 % (Table 1.3).

- Few-Shot covered slightly more key facts (0.95 vs APE 0.93).
- APE got there **26 % cheaper** ($7.58 vs $10.29 per 1,000 queries) because it carries one instruction instead of three worked examples.
- APE is therefore the best *value*. CoT produces the **deepest visible reasoning** (5.25 steps) and references the most sources (3.6), but its long reasoning did not raise correctness above 90 %.

Would we use different techniques for different complexities? Yes. Table 1.4 shows a clear split by question level:

- **Simple factual** questions: all five techniques score 100 %, so **Zero-Shot** is the right choice (2.8 s, the cheapest).
- **Medium/complex** questions: Zero-Shot drops to 60–80 %, while APE and Few-Shot stay at 100 %. Use **APE**.
- **Ambiguous** questions are where techniques differ most. APE and Few-Shot reach 100 %, CoT 80 %, and Zero-Shot and ToT only 60 %.
- **ToT is overkill here.** It used 4.1k input tokens and 45 s per ambiguous question (Table 1.5) for *lower* correctness than CoT.
    - Its evaluator picks *one* best path. On a question with several valid interpretations, that can collapse a genuine disagreement into a single answer.

    - ToT's coverage is still the highest on ambiguous questions (0.83), so its paths do find the facts. The selection step is what loses the nuance.

How CoT beat Zero-Shot (three examples, §1.5).

- On *C4 vs C3* (coverage 0.50 → 0.83), *drivers of sea-level rise* (0.33 → 0.67) and *aspartame safety* (0.50 → 0.75), Zero-Shot gave one fluent but shallow sentence.
- CoT's explicit steps forced it to extract evidence from *each* source, note where sources agree or conflict, and weigh credibility.
- That surfaced facts Zero-Shot skipped: the 44 % / 42 % ice-loss vs thermal-expansion split, JECFA keeping the 40 mg/kg ADI, and why the tabloid claim is weak.

How APE finds prompts a human might miss. The winning instruction tells the model to check whether sources *align or conflict*, explain the discrepancies, and favour academic studies (Table 1.2).

- Our hand-written CoT prompt asks for steps, not for conflict handling, and scored lower on the APE-dev set (0.25 vs 0.50).
- APE optimises the instruction *empirically* against held-aside questions. It found that, for this task, telling the model *what to reason about* (agreement and credibility) matters more than telling it *how* (numbered steps).

How research reasoning differs from simple Q&A.

- Simple Q&A has a single gold span that any technique finds: 100 % for all five.
- A research answer must cover several key facts from *different* sources, reconcile conflicting claims, weigh credibility and state uncertainty.
- That is why correctness is measured as key-fact coverage plus contradiction and uncertainty checks, and why the techniques only separate on medium, complex and ambiguous questions.

5. Task 2 — Multi-source function calling, safety & security

Tool-selection accuracy 82%, task completion 80%, parallel latency reduction 48%, defence success rate 100% with a 0% false-positive rate on benign inputs.

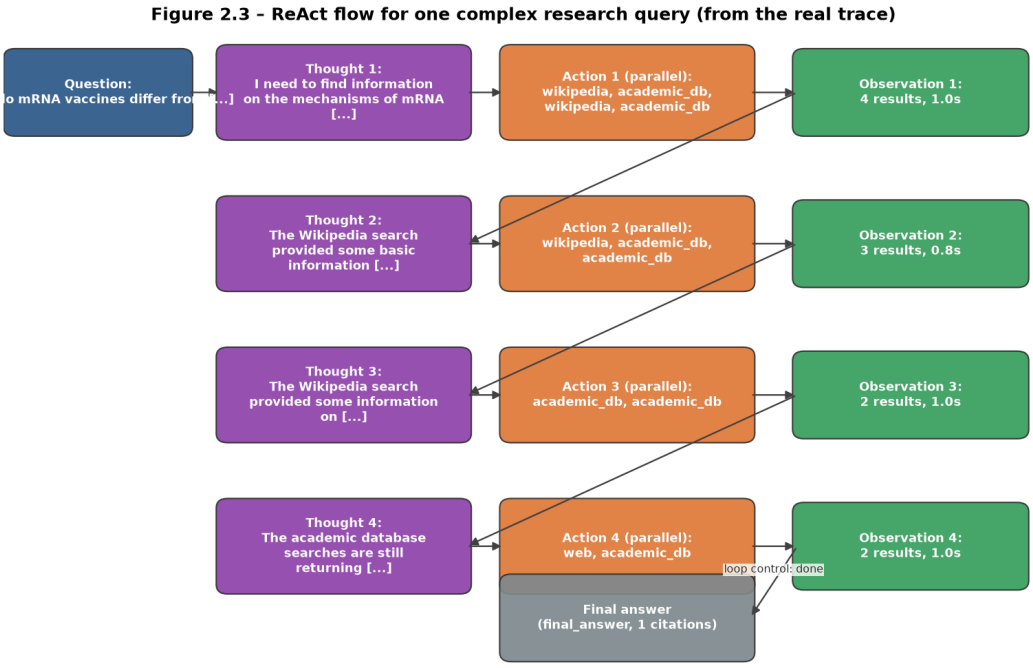


Figure 4 – ReAct flow (real trace)

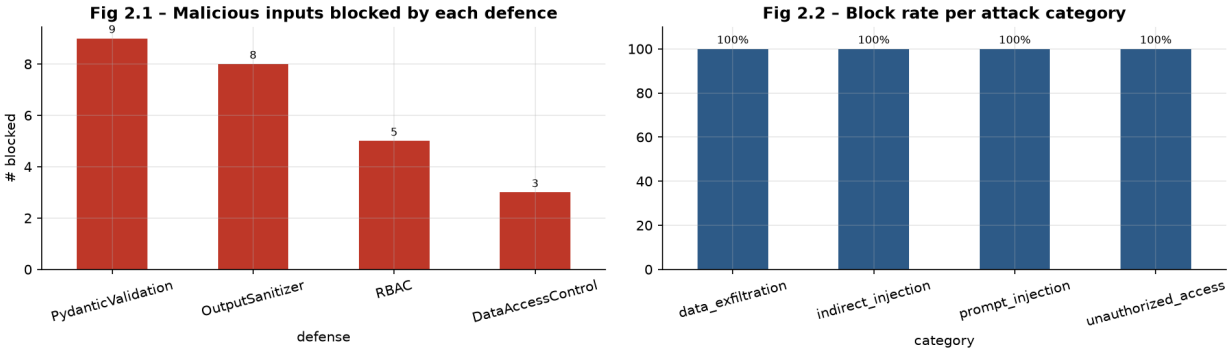


Figure 5 – Malicious inputs blocked per defence

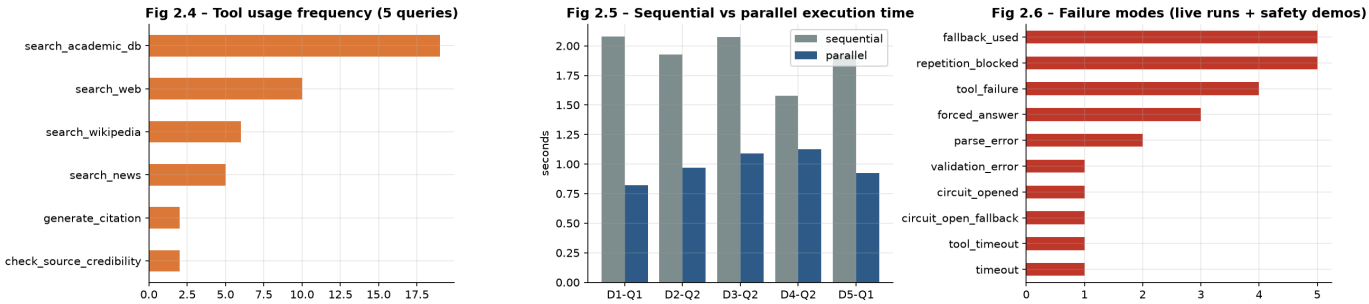


Figure 6 – Tool usage, sequential vs parallel, failure modes

How does parallel function calling improve research synthesis time?

- The three source searches are independent, so running them at once makes tool time equal to the *slowest* call instead of the *sum*.
- Across the 5 questions the measured reduction was **47.5 %** on average (28–60 %, Table in §2.3).
- The parallel run makes exactly the same calls and returns the same results, so the speed-up costs no quality.
- The gain grows with the number of sources per step. It matters less end-to-end, because LLM time still dominates: 25–135 s per question vs about 1–2 s of tools.

What was the most common failure mode?

- **Repetition** was the most common: 3 blocked repeats in the 5 live runs, vs 1 tool failure and 1 fallback.
- When the academic database returned off-topic papers (D2-Q2, D4-Q2), the agent kept re-issuing near-identical searches. The repetition detector stopped this, and D2-Q2 still used 10 steps and 19 tool calls.
- Overall, tool-selection accuracy was **82 %** and the task completion rate **80 %** (4 of 5).
- The one incomplete run (D3-Q2) stopped after generating citations without writing a full answer, which shows the agent loop also needs a check that the answer is complete, not just that a final answer exists.

Which security risk is most dangerous for a research assistant?

- **Indirect prompt injection from retrieved sources** is the most dangerous. A research assistant's job is to read untrusted text at scale, and the attacker does not need access to the user.
- In our demo the poisoned news snippet reached the undefended model, and its answer engaged with the injected "vaccines are dangerous" instruction, which our check flagged.
- With the sanitiser the instruction-like sentences were removed before the model saw them.
- The test suite shows all four defences working:
    - 25 of 25 malicious inputs were blocked: 9 by Pydantic validation, 5 by RBAC, 3 by data-access control, and 8 by the output sanitiser.
    - None of the 8 benign queries was blocked (0 % false positives).
    - The SQL-injection demo returned 43 rows, including paywalled ones, through string formatting, and 0 rows through a parameterized query.

Why is indirect injection harder to defend against than direct user input?

- **Direct input** arrives through a narrow, typed interface. A JSON schema plus Pydantic (lengths, enums, ranges, injection patterns) can validate it before any tool runs.
- **Retrieved text** is free-form *content* that the model must read in order to do its job, so it cannot be rejected by a schema.
- It also arrives through a channel the system otherwise trusts, mixed with legitimate sentences. The attacker can plant it once in a web page and reach every user.
- The defence therefore has to be layered:
    - sanitise tool outputs
    - wrap them as `<untrusted>` data
    - tell the model never to follow instructions inside evidence
    - still check the output (citations must point to retrieved, credibility-scored sources)

6. Task 3 — Memory, decomposition, retrieval, context engineering, advanced RAG

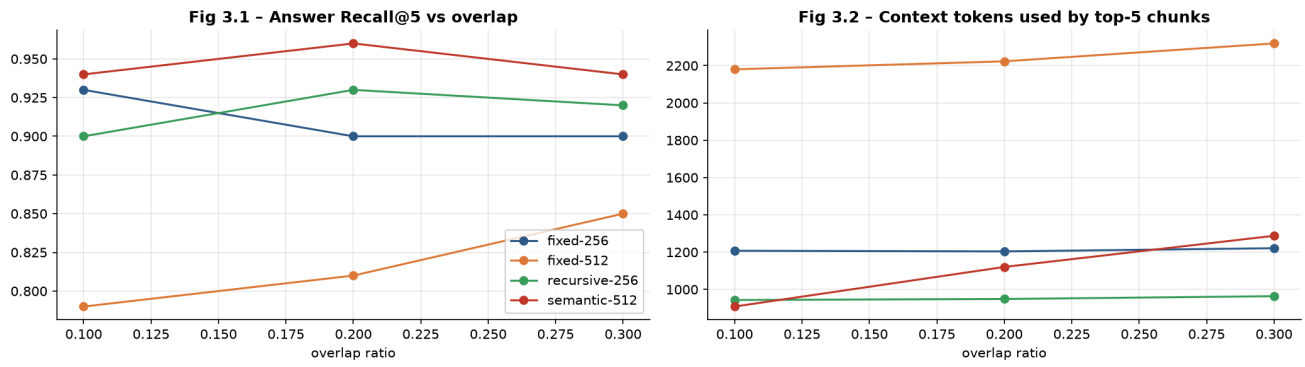


Figure 7 – Chunking strategy × overlap

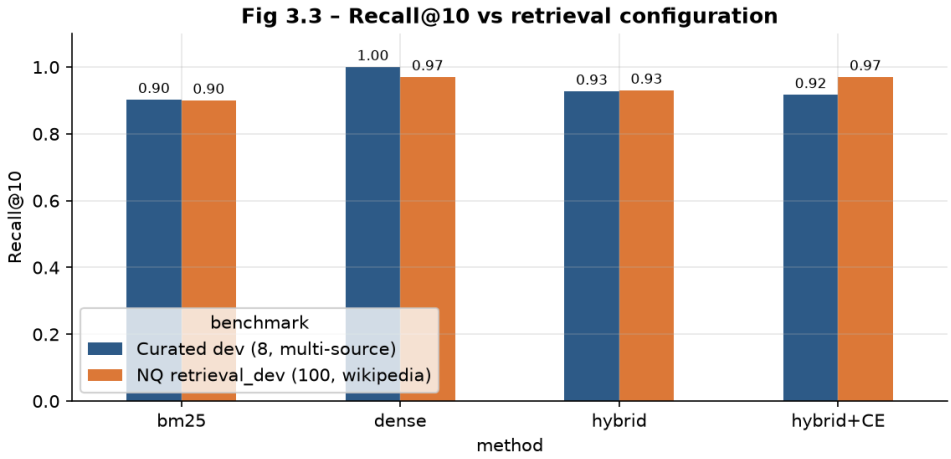


Figure 8 – Recall@10 by retrieval configuration

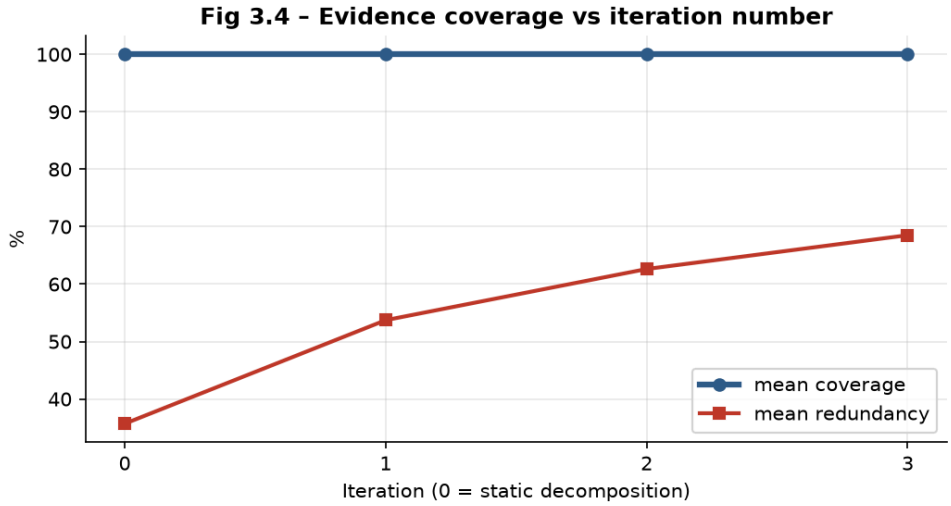


Figure 9 – Evidence coverage vs refinement iteration

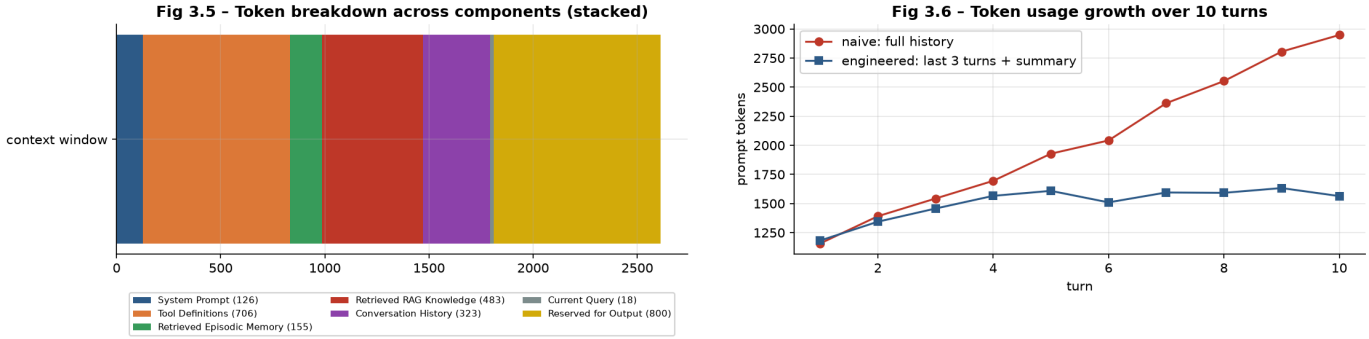


Figure 10 – Context token breakdown and growth over 10 turns



Figure 11 – Lost in the middle

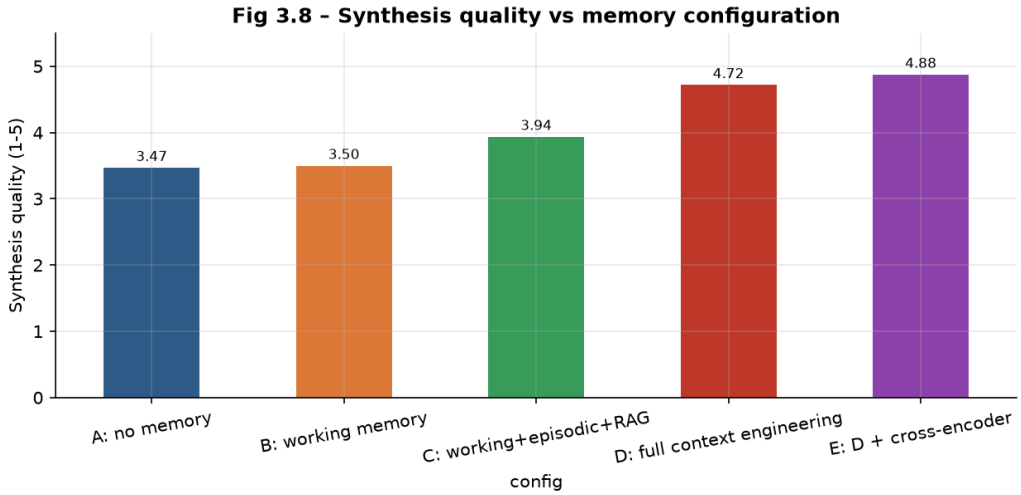


Figure 12 – Memory ablation

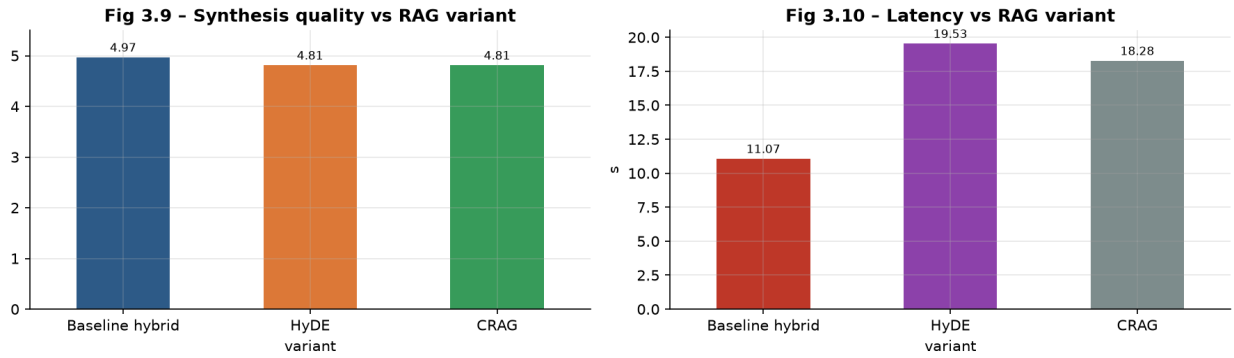


Figure 13 – HyDE / CRAG vs baseline

How does query decomposition improve evidence coverage? In the retrieval ablation over the 8 dev questions (Table 3.9):

- Coverage rises from **0.54** for one query against Wikipedia (A) to **0.98** for one query against all sources (B), and **1.00** with decomposition (C).
- Each sub-query targets one facet of the question (mechanism, effect size, criticism), so documents relevant only to that facet are retrieved too.
- Quality follows coverage at first (A 3.69 → B 4.97). C and D score about the same as B (4.91 and 4.94), so on this corpus the extra coverage from decomposition is small once all sources are searched.

Iterative refinement vs static decomposition, and the marginal benefit of each iteration.

- On our corpus, static decomposition already reaches **100 % coverage at iteration 0**, so the three refinement iterations add **0** coverage each (Table 3.6).
- They do cost a lot:
    - redundancy rises from 0.36 to 0.69
    - cumulative tokens rise from 267 to 3,495 (13×)
    - latency rises from 5.4 s to 23.5 s, about +6 s and +1.1k tokens per iteration
- This is a property of the evaluation set: each curated topic has only a handful of documents, and the first sub-queries find them all.
- On an open corpus, where the first searches miss sources, refinement is how gaps are found. That is why we keep it in the assistant, capped at 3 iterations, but would stop early when an iteration adds no new documents.

Retrieval depth vs latency.

- Deeper retrieval buys coverage only until the relevant set is exhausted. After that it just adds redundancy and latency.
- Cross-encoder re-ranking is the depth worth paying for. It adds about 0.7 s per query on NQ (1.5 s on the multi-source set) and lifts NQ Recall@5 from 0.88 to 0.94 (Table 3.4).

Which chunking strategy works best?

- **Semantic chunking (512-token cap, overlap 0.2)** had the best Answer Recall@5 (**0.96**) at about 1.1k context tokens (Table 3.3).
- Fixed 512-token chunks were clearly worst (0.79–0.85), because large blocks dilute the embedding of the answer sentence.
- Fixed-256 and recursive-256 were close behind (0.90–0.93).
- Overlap had no consistent effect within a strategy (differences ≤ 0.04), so 0.2 is a reasonable default.

Token contribution of each context component (Table 3.10 / 3.11).

- Tool definitions are the largest input component: **706 tokens, 27 %** of the 2,611-token budget. Then RAG chunks 483 (18.5 %), conversation history 323 (12.4 %), episodic memory 155 (5.9 %), system prompt 126 (4.8 %) and query 18 (0.7 %), with 800 tokens (30.6 %) reserved for output.
- By category: *required operational* context is 67.5 % and *task-relevant signal* 32.5 %. Nothing was classified as redundant: every piece had similarity ≥ 0.45 to the query.
- The operational overhead is the cost lever. Compact tool schemas save 16 % of input tokens (Task 7).
- Without engineering, the context would grow from 1,157 to 2,948 tokens over 10 turns. With last-3-turns plus summary it stays near 1.5k.

How does "lost in the middle" affect synthesis, and what mitigation worked?

- With the same 12.3k-token context, putting the relevant documents **in the middle** gave the lowest quality and coverage (3.75 / 0.50), vs **beginning** 4.62 / 0.88 (Table 3.12). The effect is real.
- Putting the best evidence **first** was the most reliable mitigation.
- "Balanced" (best documents at both ends) varied between seeds (4.50 and 3.50), so with 2 seeds we cannot claim it beats beginning-heavy.
- Our pipeline therefore orders evidence by fused score, best first.

Did cross-encoder re-ranking justify its latency? Yes.

- Config E vs D: quality **4.88 vs 4.72** (+0.16) for **+1.37 s** (+10 %) total latency (Table 3.13).
- The gain comes from a better top-5, not more tokens (+90 input tokens).

- Config C (everything included, no engineering) is the cautionary case. It has the most tokens (1,603) and the highest latency (20.8 s) but lower quality (3.94) than D. More context is not better context.

Which advanced RAG variant performed best?

- The **baseline hybrid pipeline** won: quality 4.97, coverage 0.98, 11.1 s (Table 3.14).
- **HyDE** matched its coverage but scored lower (4.81) and took 19.5 s.
- **CRAG**'s evaluator graded every retrieval "CORRECT", so its web fallback never fired. Its filtering only dropped evidence (coverage 0.88) and added 7 s.
- On an in-domain curated corpus the baseline already retrieves well, so both variants add cost without benefit. When each variant is preferable:
    - HyDE: short or vaguely worded questions whose vocabulary differs from the documents.
    - CRAG: open-domain questions where the local index can genuinely miss, so the corrective web search has something to fix.

7. Task 4 — Synthesis, conflicts, hallucination, calibration, RAG Triad

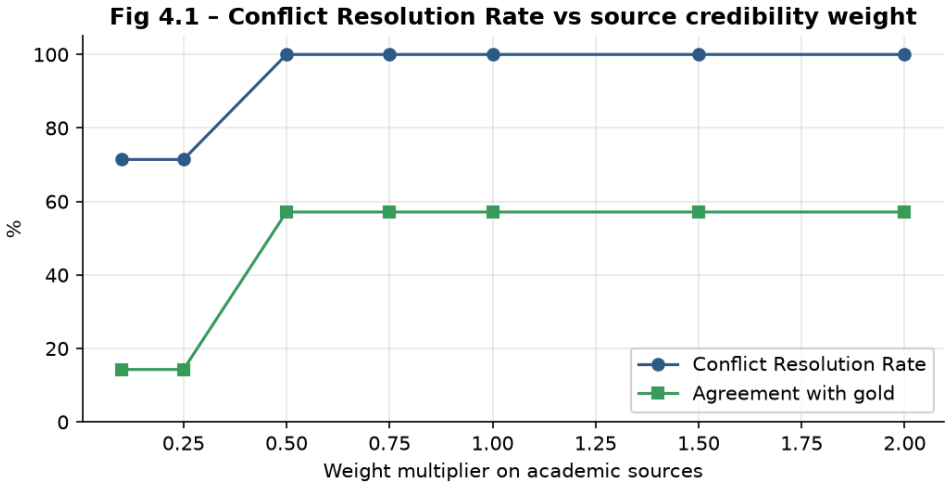


Figure 14 – Conflict resolution vs credibility weight

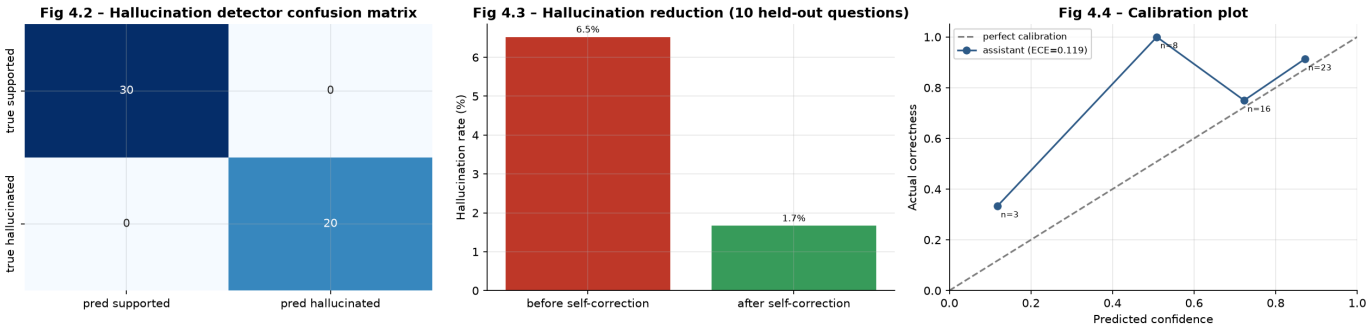


Figure 15 – Detector confusion matrix, hallucination reduction, calibration

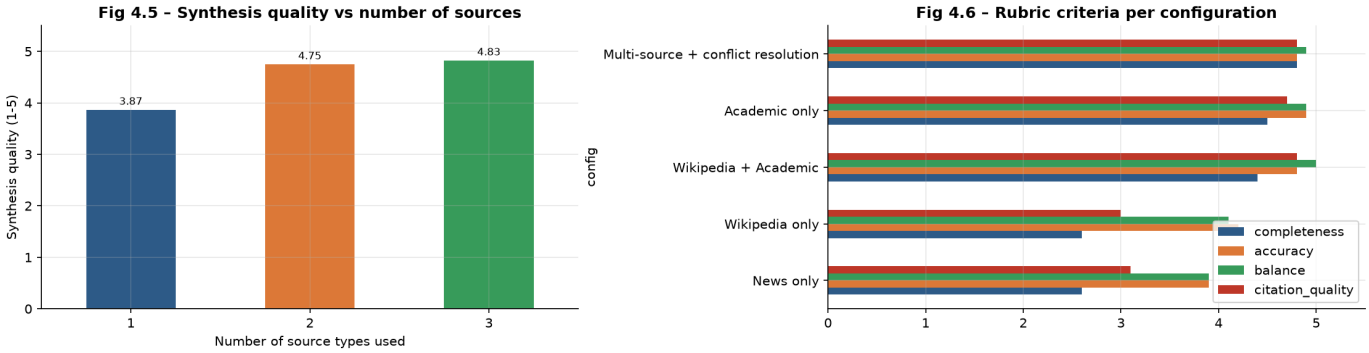


Figure 16 – Quality vs number of sources

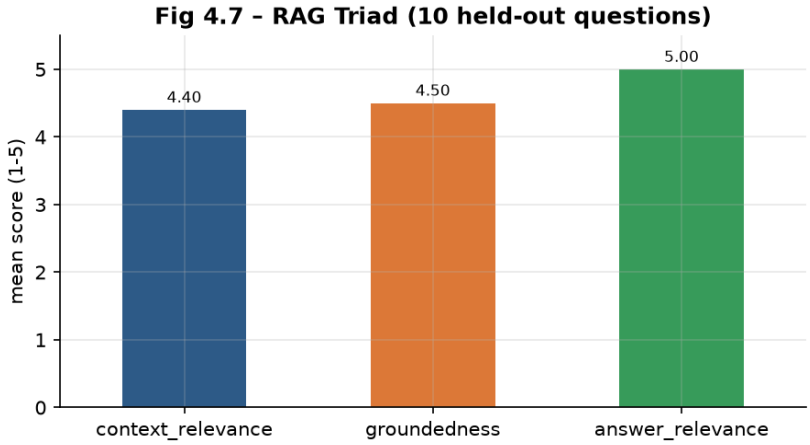


Figure 17 – RAG Triad

How does multi-source synthesis improve answer quality? Table 4.8 compares the same 10 held-out questions with different evidence:

- Multi-source with conflict resolution scored **4.82**.
- Wikipedia only scored 3.48 and news only 3.38. Their **completeness was 2.6**, because a single source rarely covers mechanism, magnitude *and* the counter-evidence.
- Academic-only (4.75) is close because the academic documents are rich. Multi-source still adds balance: the answer can say what the popular claim is *and* why the better evidence disagrees.

What are the challenges of resolving conflicting evidence?

1. **Detection.** The LLM found 7 conflicts on the dev topics but only 4 matched the adjudicated gold conflicts. The rest were complementary facets presented as disagreements.
2. **Near-ties.** Where the evidence is balanced, the honest outcome is "contested". At the 60 % threshold, majority voting left 2 of 7 conflicts unresolved.
3. **Counting voices is wrong.** Tabloids and blogs repeat one claim many times.

How does credibility weighting change the final answer?

- On D1-C1, intermittent fasting vs calorie restriction, **majority vote** picks "fasting is superior", backed by a tabloid, a blog and a forum post.
- **Credibility weighting** picks "equivalent, not superior", backed by the meta-analysis and the RCT (share 0.75). The synthesised answer changes accordingly (§4.3).
- Across the dev conflicts:
  - credibility and evidence-strength scoring resolve 100 % with **57 %** gold agreement, against 71 % and **29 %** for majority voting (Table 4.4)
  - gold agreement collapses to 14 % when academic weight is cut to ≤ 0.2× (Fig 4.1)

- In fusion, multiplying by document credibility raised precision@5 from 0.55–0.63 to 0.65–0.75 and removed low-credibility documents from the top-5 (Table 4.2).

How does self-correction reduce hallucinations?

- The verifier checks each claim against the retrieved evidence and regenerates any unsupported claim using only the evidence.
- The hallucination rate falls from **6.5 % to 1.7 %** (10.7 % → 3.2 % when the same questions run through the integrated assistant in Task 6; §6.3 explains why the two sets of numbers differ).
- On the constructed test set the detector reached F1 1.00 (30 supported, 20 fabricated claims, §4.4.1). Those fabrications are easy (off-topic facts, altered numbers), so this is an upper bound.
- Citation accuracy (does the cited source *directly* support the claim) was **83.9 %** of 56 claim–citation pairs.

Is the assistant well-calibrated? It is **under-confident**:

- mean confidence 0.72 vs claim accuracy 0.84
- **ECE 0.119** (Table 4.5)
- The worst bin is 0.4–0.6: confidence 0.51 but **100 %** correct.

Confidence vs correctness across claim types.

- Single-source claims are the most under-confident: confidence 0.70, accuracy 0.81 (Table 4.7). Our formula penalises having one source even when that source is a strong meta-analysis.
- Claims with two sources: confidence 0.85, accuracy 0.90.
- By query type (Table 4.6), factual claims (ECE 0.22) and contested claims (ECE 0.21) are about equally mis-calibrated. Contested claims show the larger gap between confidence and accuracy (0.66 vs 0.81).
- Fix: give a single high-credibility source more weight in the confidence formula, then recalibrate on the dev set (e.g. isotonic regression).

Which RAG Triad stage fails most often, and what does that tell us to fix?

- Means: context relevance **4.4**, groundedness **4.5**, answer relevance **5.0** (Fig 4.7).
- With the strict judge (it must list unsupported claims; a stage fails below 4), **grounding** is the only stage that failed: **2 of 10** queries (Table 4.9).
    - H2 added "conjugation", true background knowledge that is not in the retrieved context.
    - H9 added the evaluative phrase "making its claims less credible".
- Neither is a factual error. Both are statements the evidence does not license, so the fix is in synthesis and verification, not retrieval:
    - forbid uncited background knowledge
    - have the verifier also check evaluative qualifiers
- Context relevance has the lowest mean. The audit trail shows why: off-topic chunks from other topics occasionally fill the top-8. A per-source relevance cut-off after re-ranking would address that.

How far can we trust the judge? (Cohen's κ, §4.6.1) Our team labelled all 30 Triad items (10 questions × 3 criteria) blind to the judge's scores. Agreement is **poor**:

- κ is **0.03** unweighted, **0.10** quadratic-weighted, and **0.05** on the pass/fail decision (≥ 4). Exact agreement is 33 %.
- The disagreement is one-directional. The judge scored *higher* than the humans on 17 of 30 items and lower on only 3.
- **Groundedness** is the worst: mean 4.5 from the judge vs **3.3** from humans, with exact agreement on only 1 of 10 items.

    - The human raters marked answers down for details and background statements they could not find in the excerpt.
    - The judge mostly saw full support.
- So the Triad means above are an *optimistic* upper bound, especially groundedness. By the human scores, grounding is weaker than the judge suggests, which reinforces the fix above: forbid uncited background knowledge.
- Before relying on the judge in production we would add few-shot human-scored examples to the judge prompt, use a stronger or ensemble judge, and recalibrate it on a larger human-labelled set.

8. Task 5 — Self-Instruct, DPO, PEFT, alignment, paradigms

Self-Instruct: 50 generated → 40 kept after filtering (removed: {'duplicate (ROUGE-L>=0.7)': 0, 'heuristic': 2, 'low LLM quality': 8}). DPO loss = 0.1777. LoRA trainable parameters = 0.249% of 7B. 70B memory (P×B×1.2): {'FP32': 336.0, 'BF16': 168.0, 'INT8': 84.0, 'INT4 / NF4': 42.0}.

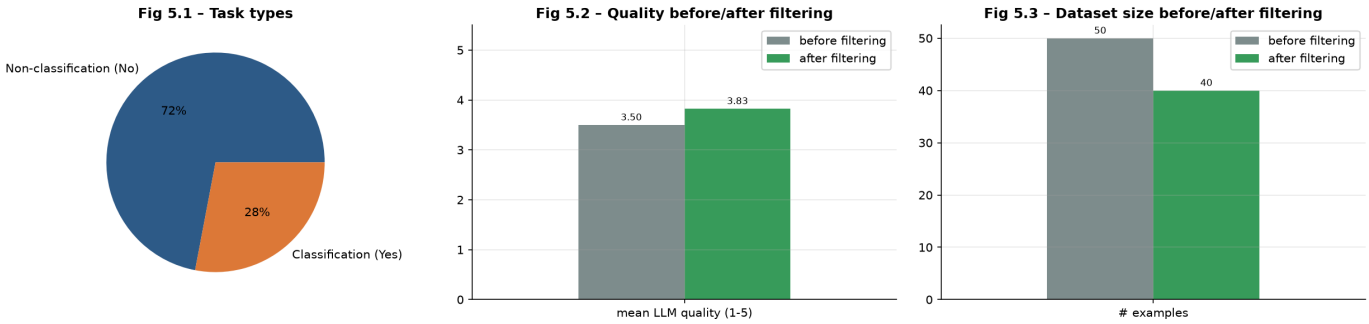


Figure 18 – Self-Instruct task types and filtering

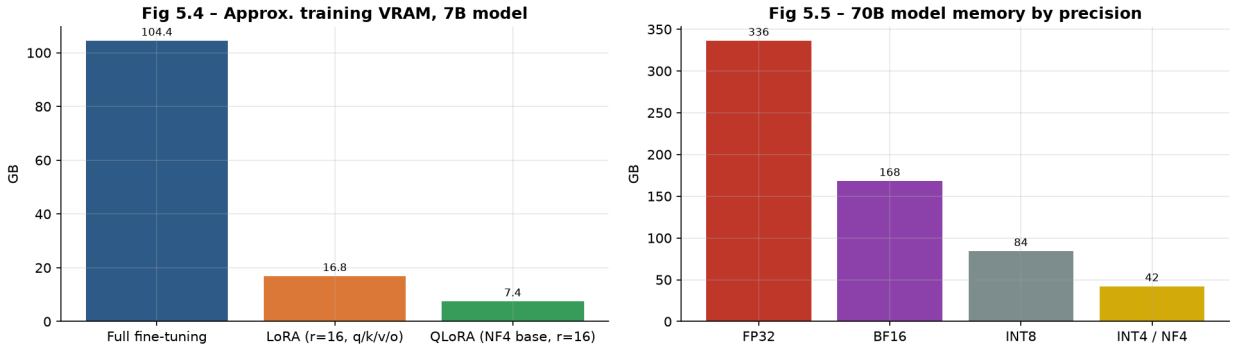


Figure 19 – PEFT memory

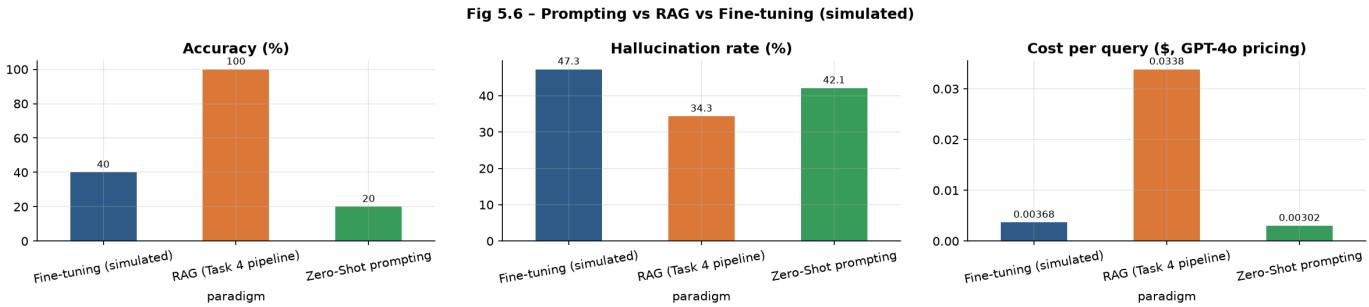


Figure 20 – Prompting vs RAG vs fine-tuning

Figure 5.7 - Decision tree: Prompting vs RAG vs Fine-tuning for a research task

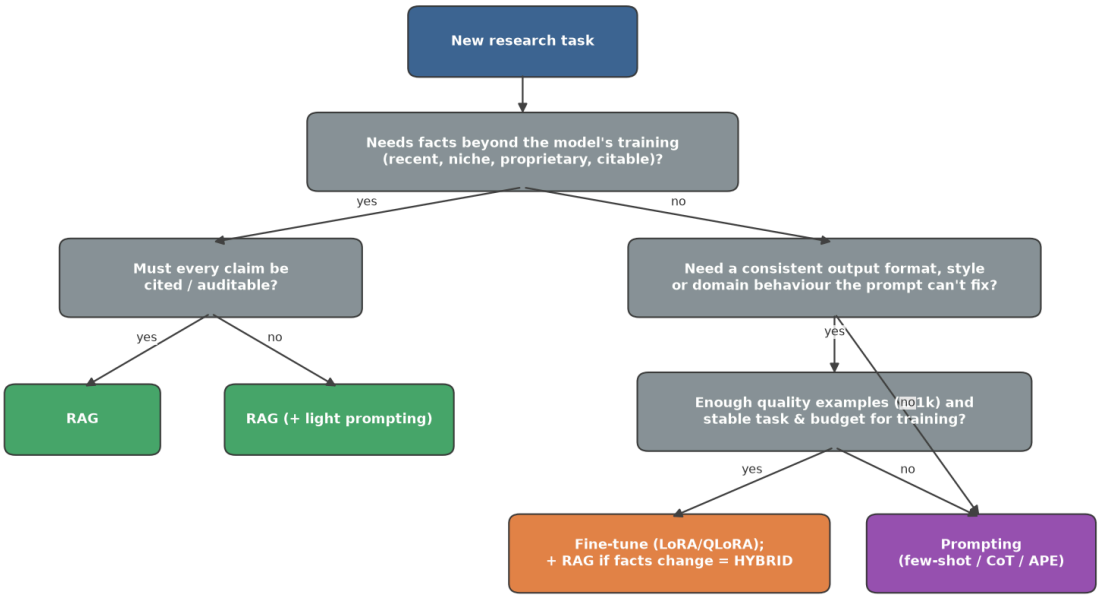


Figure 21 – Decision tree

What are the risks of self-generated research tasks?

- **Inconsistent labels.** Output-first classification produced 13 distinct labels, including case variants such as "neutral" and "Neutral" (Table in A.5). Generated label spaces drift unless they are constrained.
- **Lost diversity.** Unique leading verbs fell from 15 to 12 after filtering.
- **Quality.** 8 of 50 examples were removed as low-quality and 2 by heuristics, leaving 40 (mean judged quality 3.5 → 3.82).
- **Self-reinforcement.** Because the same model family writes and filters the data, its blind spots and factual errors (invented studies in the inputs) can be reinforced instead of corrected.
- Output-first generation helps balance classes (we choose the label first, then generate an input for it), but it needs a fixed label set.

Why does DPO not need a separate reward model?

- Under the Bradley-Terry preference model, the optimal RLHF policy has a closed form. The reward becomes an *implicit* function of the policy: $r = \beta \cdot \log(\pi_\theta / \pi_{ref})$.
- DPO therefore optimises preferences directly with a classification-style loss on (preferred, rejected) pairs. There is no reward model to train and no PPO rollouts.
- In our worked example the policy already raised the balanced summary (ratio 1.71) and lowered the one-sided one (0.33). The resulting loss is **0.178**, well below $\ln 2 = 0.693$ for an untrained policy.

How does QLoRA make fine-tuning large models resource-efficient? It combines three ideas:

- a frozen **4-bit NF4** base model
- **double quantisation** of the scaling constants, saving about 0.37 bits per parameter
- **paged optimisers**, which absorb memory spikes

Only the LoRA adapters are trained, 0.25 % of a 7B model. In our estimates, a 7B fine-tune drops from about 104 GB of GPU memory (full) to about **7.4 GB** (QLoRA). For a 70B model the weights alone need 336 GB in FP32 but 42 GB in NF4, which is one 80 GB GPU instead of five.

Why does Zero-Shot fail at synthesising conflicting evidence?

- Zero-Shot has no sources, so it cannot see that sources disagree, weigh their credibility or cite them.
- It answers from the most common view in its training data. On the 5 contested questions it was correct only **20 %** of the time with key-fact coverage 0.38.
- RAG with conflict resolution was correct **100 %** of the time with coverage 0.84 (Table 5.5).
- Simulated fine-tuning (40 %) learned the *style* of a balanced synthesis, but without evidence it still could not supply the facts.
- The verifier flags many claims in all three paradigms (34–47 %), because it checks against the topic's credible documents only. Claims drawn from Wikipedia or news count as unsupported there. The *ranking* (RAG lowest) is what matters.

Why is a hybrid RAG + fine-tuning approach useful? They fix different failures:

- **RAG** fixes *knowledge*: fresh, attributable, conflict-aware.
- **Fine-tuning** fixes *behaviour*: the claim ↔ citation format, balanced framing and explicit uncertainty, learned once instead of carried in a long prompt.
- RAG costs about 10× more per query than Zero-Shot ($0.034 vs $0.003) and is about 5× slower. A LoRA-tuned model can shorten the instructions RAG needs to carry, and route easy questions without retrieval.

9. Task 6 — Integration, multi-turn, audit trail, evaluation

Figure 6.1 - Multi-turn research conversation flow

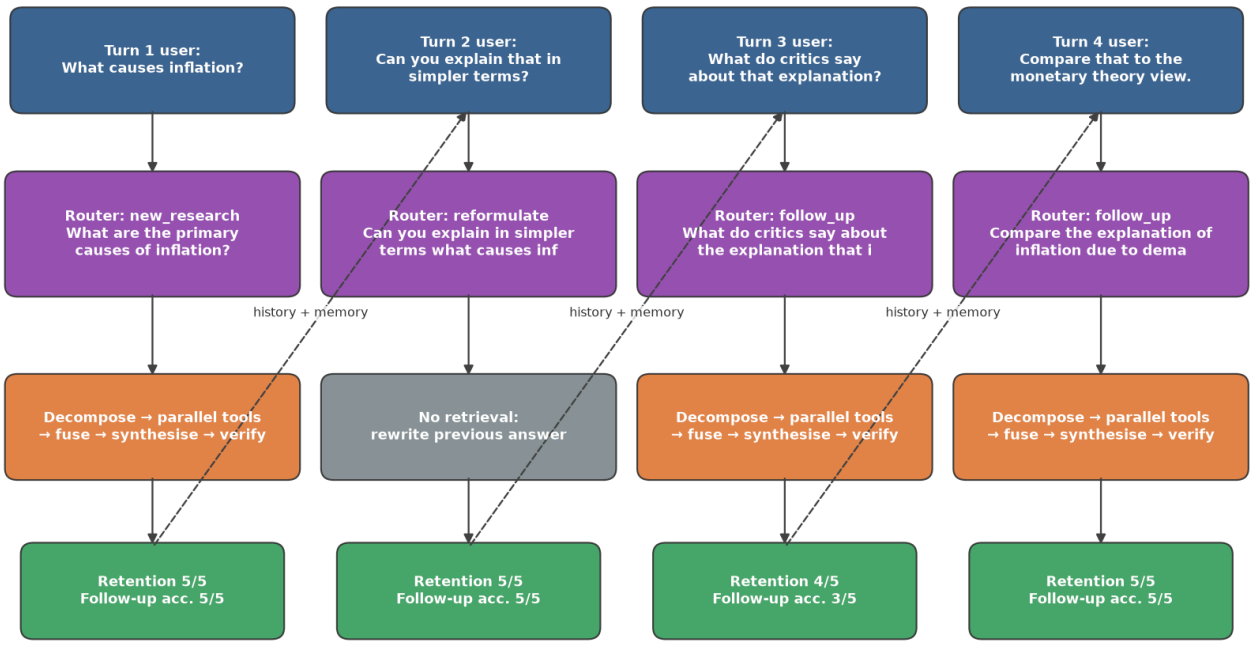


Figure 22 – Multi-turn flow

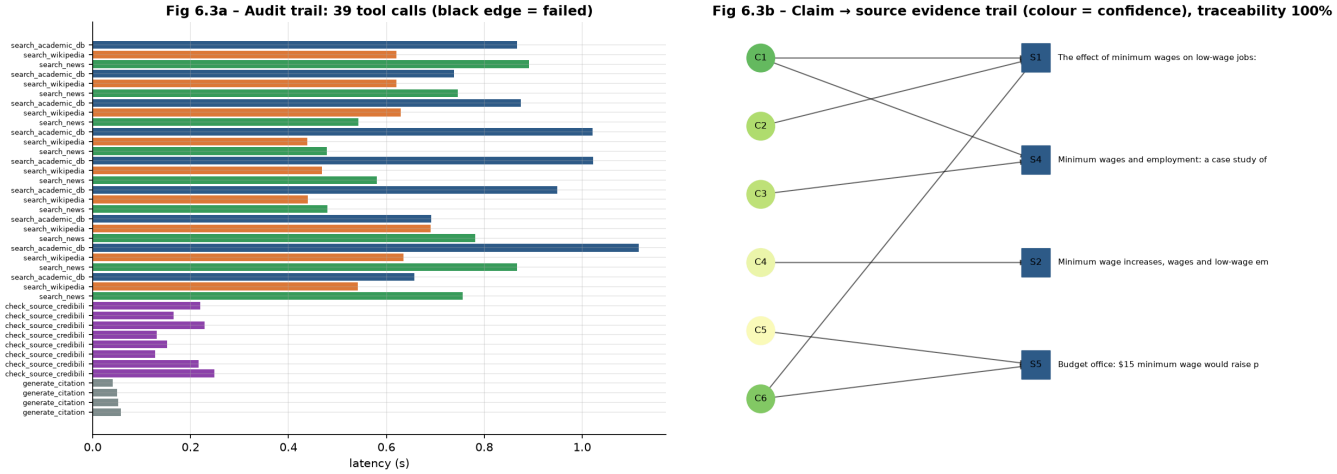


Figure 23 – Audit trail

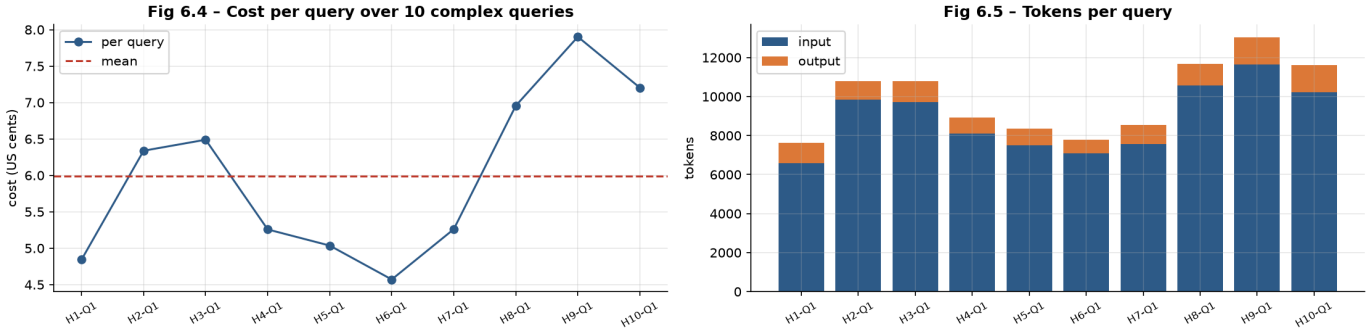


Figure 24 – Cost and tokens per query

How does a multi-turn research conversation differ from single-turn Q&A?

- Later turns depend on earlier ones: "that explanation" and "that" must be resolved against the history.
- The router must also decide *what kind* of turn it is:
  - Turn 2 was routed as **reformulate**, which reuses the evidence: 8 s instead of 56 s.
  - Turns 3–4 were **follow-ups** with fresh retrieval, after the turn was rewritten as a standalone query.
- Context retention was 5/5/4/5 and follow-up accuracy 5/5/**3**/5 (Table 6.1).
- The weak turn is "What do critics say…": the rewritten query was right, but the answer mostly restated demand-pull/cost-push instead of presenting critiques. A follow-up needs retrieval aimed at the *new* facet, not only the old topic.

How does the audit trail improve transparency and trust?

- For every query, the JSON record stores:
  - the original question
  - the static and refinement sub-queries
  - every tool call (arguments, timestamp, latency, output size)
  - every source with its credibility
  - every claim with its evidence snippet, supporting source and confidence
- A reviewer can therefore replay *why* the assistant said something, not just what it said. Claims built on weak sources (credibility < 0.5) are visible at a glance.

Can a user trace every claim to its supporting evidence? Almost:

- Claim-to-source traceability is **96.75 %** over the 10 complex queries, and 100 % for the rendered H7 report.
- The remaining claims are synthesis sentences whose citation did not point to a retrieved source.

- Traceability is lower on the 2 single-source queries (90 %) than on multi-source ones (98 %).

What is the most critical component for synthesis quality? Evidence selection. That means multi-source retrieval with credibility-aware fusion and re-ranking.

- It produces the largest measured jumps: one query against Wikipedia vs all sources, 3.69 → 4.97 (Table 3.9); single vs multi-source synthesis, 3.48 → 4.82 (Table 4.8).
- Engineering what reaches the prompt matters next: Config C → D, 3.94 → 4.72.
- The breakdown in Table 6.2 points the same way. Citation accuracy is 84 % on contested queries but 57 % on factual ones, and hallucination after correction is 1.6 % for multi-source vs 10 % for single-source queries.

Is the system production-ready? What would we fix first? Not yet.

- Quality is good: synthesis 4.62, conflict resolution 100 %, hallucination 3.2 % after correction, task completion 100 % complex and 98 % NQ-50, NQ accuracy 86 %.
- **Latency is the first fix.** P50 is 66 s and P95 88 s for a complex query (8.2 sequential LLM calls), which is too slow for interactive use. We would:
    - run conflict detection in parallel with drafting
    - cap refinement when it adds nothing (Task 3 showed 0 coverage gain)
    - route helper calls to a small model
    - stream the answer
- **Citation accuracy on factual queries (57 %)** is next, followed by a better-aligned judge (human κ is only 0.03–0.10, §4.6.1) and a real (non-synthetic) source corpus.
- At GPT-4o prices a complex query costs **$0.060**, about 8.9k input and 1.0k output tokens.

10. Task 7 — Architecture decisions & cost engineering

Design choice	Selected option	Justification (from our experiments)
Retrieval method	Hybrid RRF (BM25 + dense) + cross-encoder re-ranking	Table 3.4: NQ Recall@10 BM25 0.90 / dense 0.97 / hybrid 0.93; curated multi-source Recall@10 0.90 / 1.00 / 0.93. Dense alone has the best Recall@10 (dense), but only the top-5 reaches the prompt: NQ Recall@5 is highest for hybrid + cross-encoder (0.94 vs dense 0.90), and BM25 keeps exact names/numbers retrievable. On the small curated set dense is ahead (Recall@5 0.87 vs 0.82), so the two are close; we keep hybrid + CE for robustness and accept ~0.7s re-ranking latency.
Chunking strategy	Semantic 512 tokens, overlap 0.2	Table 3.3 / Fig 3.1: best Answer Recall@5 = 0.96 among 12 strategy × overlap configurations; ties broken by fewer context tokens.
Re-ranking	Cross-encoder ON	Table 3.13 Config E vs D: quality 4.88 vs 4.72 (Δ +0.16), latency 14.41s vs 13.04s (+1.37s); Table 3.4: Recall@5 hybrid 0.88 → +CE 0.94.
Advanced RAG variant	Baseline hybrid	Table 3.14: quality Baseline hybrid 4.97, HyDE 4.81, CRAG 4.81; coverage Baseline hybrid 0.97, HyDE 0.97, CRAG 0.88; latency Baseline hybrid 11.1s, HyDE 19.5s, CRAG 18.3s
Memory design	Combined (working + episodic + semantic) with full context engineering	Table 3.13: quality A 3.47, B 3.50, C 3.94, D 4.72, E 4.88; best = E: D + cross-encoder.
Agent loop	ReAct with parallel function calling	§2.3: parallel calls cut tool latency by 48% with identical results (same calls, same outputs); tool-selection accuracy 82%, task completion 80%.
Synthesis strategy	Multi-source with conflict resolution	Table 4.8: quality News only 3.38, Wikipedia only 3.48, Wikipedia + Academic 4.75, Academic only 4.75, Multi-source + conflict resolution 4.83; conflict gold-agreement (credibility) 57% vs majority 29% (Table 4.4).
Hallucination control	Verification ON (+ self-correction)	Fig 4.3: hallucination rate 6.5% → 1.7% after the verification + self-correction pass; detector F1 1.00 (Fig 4.2).
RAG Triad weak stage	Groundedness (grounding)	Table 4.9: stage that failed most often (score < 4) = grounding in 2 of 10 queries; means context_relevance 4.40, groundedness 4.50, answer_relevance 5.00.

Strategy	Cost / query ($)	Monthly cost ($)	Savings vs A ($/month)	Reduction (%)
A: Flat GPT-4o	0.01750	5,250.00	0.00	0.0
B: Routing 80% Small / 15% MoE / 5% Dense	0.00140	425.25	4,824.75	91.9

Strategy	Baseline A ($)	Monthly cost ($)	Savings ($)	Reduction (%)
Prompt caching (50% discount on cached prefix = 46% of input)	5,250.00	4,560.88	689.12	13.1
Prompt caching (90% discount on cached prefix = 46% of input)	5,250.00	4,009.58	1,240.42	23.6
Model routing (80/15/5)	5,250.00	425.25	4,824.75	91.9
Context reduction (-16% input tokens)	5,250.00	4,781.20	468.80	8.9
All three combined	5,250.00	297.96	4,952.04	94.3

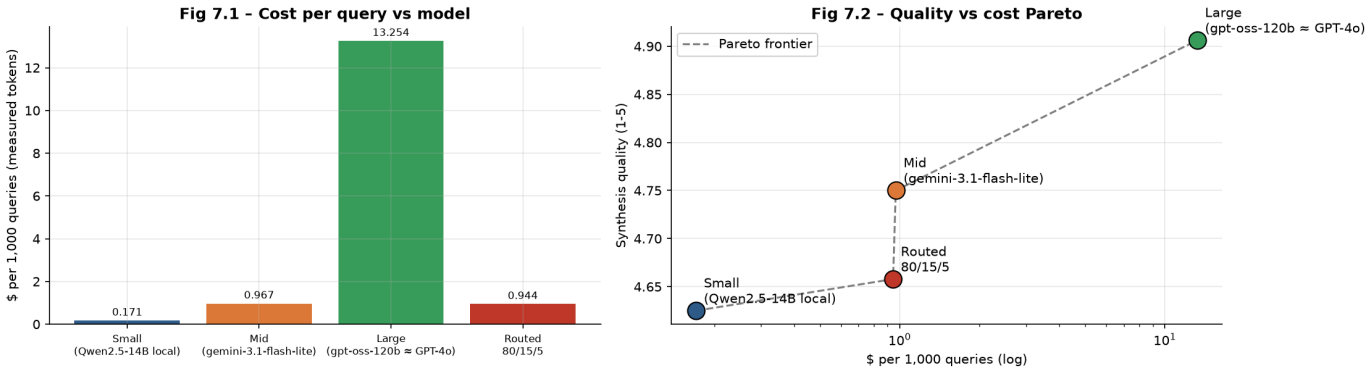


Figure 25 – Model comparison dashboard

Final recommendation (each choice is justified by Table 7.1 and the experiments it cites):

Component	Recommendation	Evidence
RAG strategy	Hybrid BM25 + dense with RRF, **cross-encoder re-ranking**, semantic 512-token chunks (overlap 0.2), baseline pipeline (no HyDE/CRAG on an in-domain corpus), static decomposition with refinement capped and stopped early	Recall@5 0.94 with CE; chunking Recall@5 0.96; baseline 4.97 vs HyDE/CRAG 4.81; refinement +0 coverage for 13× tokens
Conflict resolution	**Credibility-weighted decision** (evidence strength as tie-break), "contested" below a 60 % share	Gold agreement 57 % vs 29 % for majority vote; changes the answer on D1-C1
Prompting	**APE-selected instruction** for synthesis; Zero-Shot for simple factual look-ups; no ToT in production	APE 100 % correct at $7.58/1k; Zero-Shot 100 % on simple questions; ToT 4× the cost for 60 % on ambiguous questions
Memory design	**Working (N = 10 + rolling summary) + episodic top-2 + RAG top-3 with full context engineering**	Config E 4.88 vs C 3.94 with fewer tokens; history stays near 1.5k tokens over 10 turns
Model routing	**80 % small / 15 % MoE / 5 % dense**, with a complexity router that sends contested multi-source synthesis to the dense tier	Routed quality 4.66 at $0.94/1k vs dense-only 4.91 at $13.25/1k; $425/month vs $5,250 for 10k queries/day

What is the single most impactful change to research synthesis quality?

- **Retrieving from multiple, credibility-weighted sources instead of one.**
- Going from Wikipedia-only to multi-source retrieval raised synthesis quality from **3.69 to 4.97** (+1.28, Table 3.9).
- It took single-source synthesis from 3.48 to **4.82** (Table 4.8).
- No other change came close. Context engineering (+0.78) and cross-encoder re-ranking (+0.16) come next.

How does model routing affect cost while keeping quality acceptable?

- Routing exploits the fact that most research traffic is easy: the router sent **76 %** of NQ queries down the simple path.
- On identical evidence, the tiers scored small (local Qwen2.5-14B) 4.63, MoE 4.75 and dense (gpt-oss-120b) 4.91 (Table 7.4).
- The 80/15/5 mix gives **4.66** quality at **93 % lower cost** than dense-only. That is a 0.25-point drop, and it lands mostly on simple questions where every tier is already correct.
- At the brief's token assumptions, routing cuts the monthly bill from **$5,250 to $425** (−91.9 %).

- Combining routing with prompt caching and compact tool schemas reaches **$298** (−94.3 %, Table 7.3).
- The risk is mis-routing a contested question to the small tier. The router should therefore err towards the larger model whenever conflicts are detected.

11. Conclusion, limitations and future work

Key observations.

- We built an end-to-end research assistant covering the full pipeline: decomposition, parallel multi-source tool calls through a ReAct agent with six safety mechanisms, credibility-weighted fusion and re-ranking, conflict resolution, cited synthesis, verification with self-correction, calibrated confidence, memory, and an audit trail.
- On 10 held-out complex questions it reached synthesis quality **4.62/5**, conflict resolution **100 %** (67 % agreement with adjudicated answers), citation accuracy **74 %**, and hallucination **10.7 %** → **3.2 %** after self-correction.
- It also reached claim-level ECE **0.19**, task completion **100 %**, and traceability **97 %**.
- On 50 held-out Natural Questions accuracy was **86 %** with **98 %** completion. Cost was $0.060 per complex query at GPT-4o prices.

Strengths.

- Every design decision is backed by an ablation in this notebook. The biggest wins were multi-source retrieval (+1.28 quality) and credibility weighting (gold agreement 29 % → 57 %).
- The security layer blocked 25 of 25 attacks with no false positives.
- The run is fully reproducible:
  - seed 42
  - temperature 0
  - prompts without wall-clock values
  - an SQLite response cache that replays all 1,446 LLM calls with no live calls
- All generation ran on a local open model (Qwen2.5-14B on an Intel Arc A770), and one judge from a different model family scored everything.

Limitations.

- The multi-source corpus is largely synthetic and small, so retrieval is easier than on the open web. This is why iterative refinement and CRAG showed no benefit here.
- The judge is a single LLM, and it agrees poorly with our human labels (κ 0.03, quadratic 0.10 on 30 items, §4.6.1). It is clearly more lenient, especially on groundedness (4.5 vs 3.3), so the judge-based scores are optimistic.
- The verifier is the generator model, tested only on easy fabrications.
- Latency (P50 66 s, P95 88 s) is too high for interactive use.
- Confidence is under-confident for single-source claims.

Future work.

- Connect real sources (PubMed, Semantic Scholar, news APIs) and re-run the same harness.
- Add a dedicated NLI verifier and harder perturbation tests.
- Recalibrate confidence (isotonic) on the dev set.
- Stop refinement early when an iteration adds no documents, parallelise the helper calls and stream answers.

- Fine-tune a small model with QLoRA on the filtered Self-Instruct data plus DPO preferences, so the 80 % small tier writes in the research-synthesis format without long instructions.

12. References

Kwiatkowski et al. (2019) Natural Questions, TACL · Wei et al. (2022) CoT · Yao et al. (2023) ToT; ReAct · Zhou et al. (2023) APE · Cormack et al. (2009) RRF · Reimers & Gurevych (2019) SBERT · Johnson et al. (2019) FAISS · Liu et al. (2023) Lost in the Middle · Gao et al. (2023) HyDE · Yan et al. (2024) CRAG · Wang et al. (2023) Self-Instruct · Rafailov et al. (2023) DPO · Hu et al. (2022) LoRA · Dettmers et al. (2023) QLoRA · Shao et al. (2024) GRPO · Bai et al. (2022) Constitutional AI · Chen et al. (2024) SPIN · Lightman et al. (2023) PRM · Guo et al. (2017) Calibration · Zheng et al. (2023) LLM-as-a-Judge · Qwen Team (2024) Qwen2.5 · Google Gemini and Groq API documentation (Nov 2026) · AIMLCZG521 course sessions 4–12. The full list is in the notebook.

Appendix A. Virtual Lab environment

The project workspace is set up in the prescribed BITS Virtual Lab (Prayogshala) under student account `s2025ae05434`, at `~/workspace/conversational_ai_assignment2`. The terminal lists the repository tree. The editor window shows `ra/agent.py` (the ReAct agent, Task 2) open in GVIM. The session path and access token in the address bar are redacted.

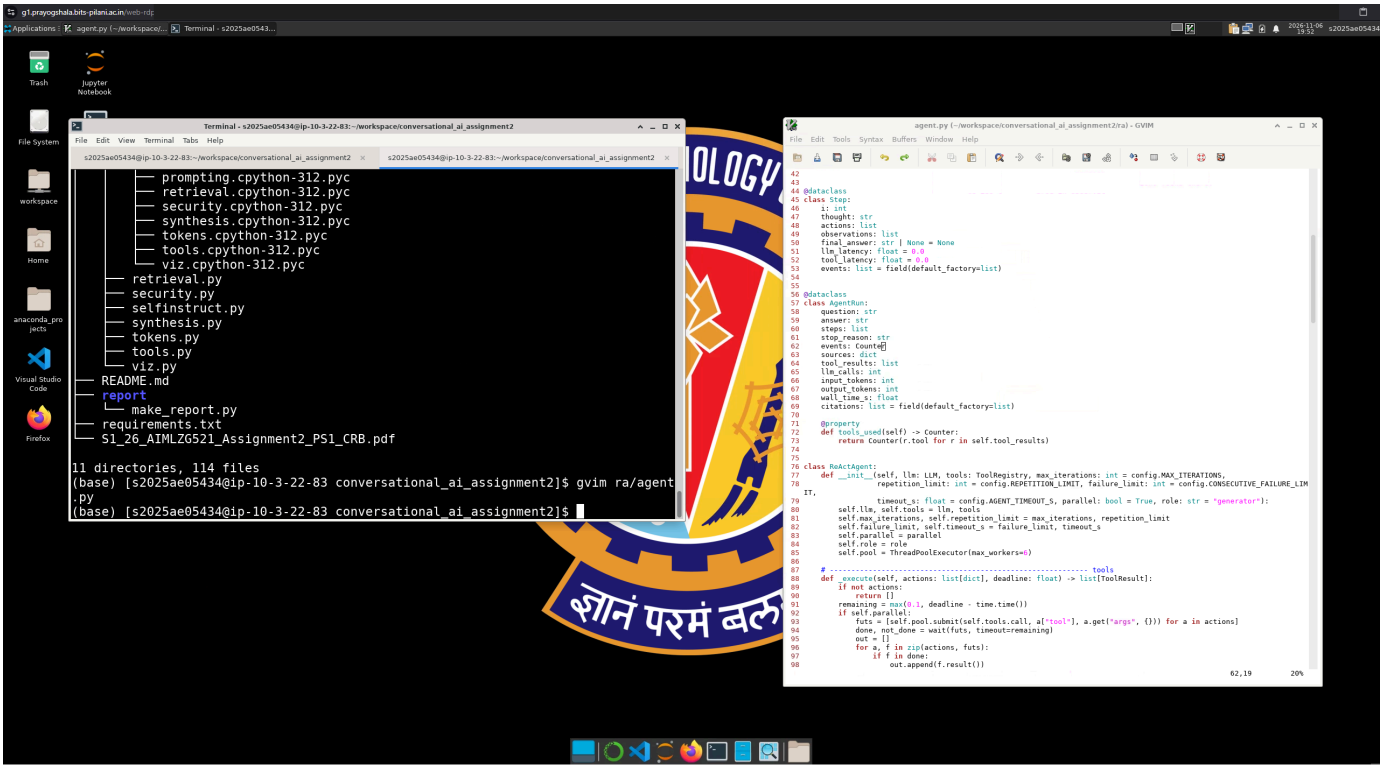


Figure A1 – The project workspace in the BITS Virtual Lab (Prayogshala), 6 Nov 2026

The second screenshot shows the submission notebook open in JupyterLab in the same Virtual Lab, on the Python 3 (ipykernel) kernel, with the full workspace in the file browser.

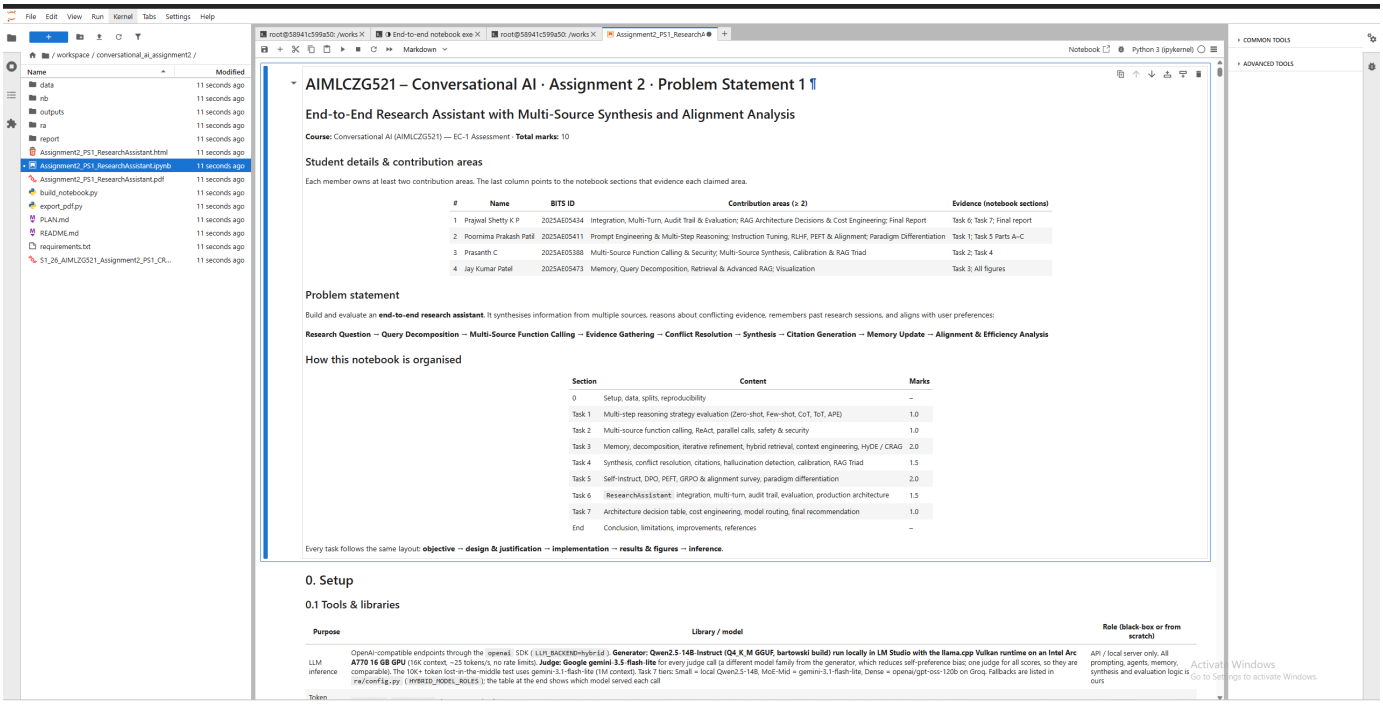


Figure A2 – The submission notebook open in JupyterLab in the BITS Virtual Lab (Prayogshala), 6 Nov 2026